ECE 499/599 | Digital VLSI Design Automation | Winter 2025 Project

Christopher Rockett,*rockettc@oregonstate.edu*
David Velasco, *velascod@oregonstate.edu*

March 20, 2026

1 Introduction

In this paper, the implementation of a serial-input-serial-output multiplier is designed and developed using a modular approach. This system takes serial input, converting an 8 individual input bits into parallel form which is then fed through a 4-bit multiplier. The resulting 8-bit output is then converted back into a serial string.

The architecture of this multiplier includes 2 forms of shift registers, a 4-bit multiplier, and a finite state machine controller. The shift registers are in charge of serial-input to parallel-output and parallel-input to serial output.

The operation of this serial-to-serial multiplier is controlled by a finite state machine (FSM). The FSM coordinates the loading input for both the serial-in parallel-out and parallel-in serial-out modules and the multiplication execution.

The design of this multiplier is done in Verilog. Core blocks are simulated in Cadence SimVision in order to validate logic. Further verification is done by synthesizing the design using 130 nm CMOS standard cell technology in Cadence Genus. Post-synthesis results are then used to evaluate power and timing performance to validate design functionality in order to move on and commence place and route of the serial-to-serial multiplier design.

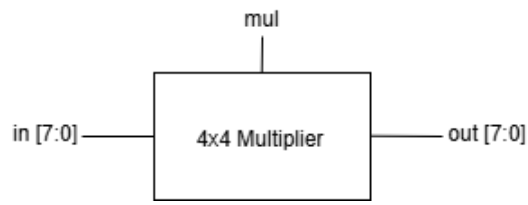
2 Core Functional Blocks

2.1 4 x 4 Multiplier

2.1.1 Module Description:

A module that performs multiplication based on two 4-bit numbers. A single 8-bit input is split into two parts: the lower 4 bits and upper 4-bits to create the two operands. The multiplier outputs a 8-bit product when the input **mul** signal is high. When **mul** is low, the output is not updated and remains at 0.

2.1.2 Symbol:



2.1.3 Verilog Module:

```
module multiplier4 (input [7:0] in,
                    input mul,
                    output reg [7:0] out);

        wire [3:0] A, B, c;
        wire [7:0] S1, S2, S3, S4, ppo1, ppo2, ppo3, ppo4;
        assign A = in[7:4];
        assign B = in[3:0];
        assign S1[7:4] = 4'b0000;
        assign S2[7:5] = 3'b000;
        assign S2[0]   = 1'b0;
        assign S3[7:6] = 2'b00;
        assign S3[1:0] = 2'b00;
        assign S4[7]   = 1'b0;
        assign S4[2:0] = 3'b000;

        and2 and2_00 (.a(A[0]), .b(B[0]), .y(S1[0]));
        and2 and2_01 (.a(A[1]), .b(B[0]), .y(S1[1]));
        and2 and2_02 (.a(A[2]), .b(B[0]), .y(S1[2]));
        and2 and2_03 (.a(A[3]), .b(B[0]), .y(S1[3]));

        and2 and2_10 (.a(A[0]), .b(B[1]), .y(S2[1]));
        and2 and2_11 (.a(A[1]), .b(B[1]), .y(S2[2]));
        and2 and2_12 (.a(A[2]), .b(B[1]), .y(S2[3]));
        and2 and2_13 (.a(A[3]), .b(B[1]), .y(S2[4]));

        and2 and2_20 (.a(A[0]), .b(B[2]), .y(S3[2]));
        and2 and2_21 (.a(A[1]), .b(B[2]), .y(S3[3]));
        and2 and2_22 (.a(A[2]), .b(B[2]), .y(S3[4]));
        and2 and2_23 (.a(A[3]), .b(B[2]), .y(S3[5]));

        and2 and2_30 (.a(A[0]), .b(B[3]), .y(S4[3]));
        and2 and2_31 (.a(A[1]), .b(B[3]), .y(S4[4]));
        and2 and2_32 (.a(A[2]), .b(B[3]), .y(S4[5]));
        and2 and2_33 (.a(A[3]), .b(B[3]), .y(S4[6]));

        rippleadder ra_0 (.a({1'b0, S1[4:1]}), .b(S2[4:1]), .cin(1'b0), .s(ppo1[4:1]), .cout(c[0]));
             // first 2 multiplication lines added
        rippleadder ra_1 (.a({1'b0, ppo1[4:2]}), .b(S3[5:2]), .cin(1'b0), .s(ppo2[5:2]), .cout(c[1]));
             // next 2 multiplication added
        rippleadder ra_2 (.a({1'b0, ppo2[5:3]}), .b(S4[6:3]), .cin(1'b0), .s(ppo3[6:3]), .cout(c[2]));
             // next 2 multiplication added
        rippleadder ra_3 (.a({c[2], ppo3[6:4]}), .b({1'b0, c[1], c[0], 1'b0}), .cin(1'b0), .s(ppo4[7:4]), .cout(
             c[3])); // carries added

        always @(*) begin
                if (mul == 1)
                out = {ppo4[7:5], ppo3[4:3], ppo2[2], ppo1[1], S1[0]};
                else
                out = 8'b00000000;
        end
endmodule
```

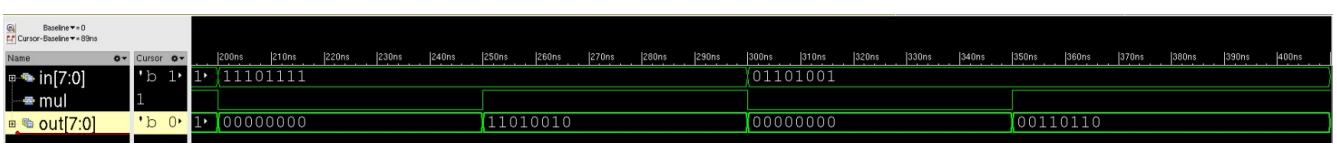
2.1.4 Verilog Testbench:

```
module multiplier4_tb;
        reg [7:0] in;
        reg mul;
        wire [7:0] out;
        multiplier4 mul_00 (.in(in), .mul(mul), .out(out));

        initial begin
                in = 8'b10101010; // 1010 * 1010
                mul = 0;

                #50

                mul = 1;

                #50 in = 8'b11111111; // 1111 * 1111
                mul = 0;

                #50

                mul = 1;

                #50 in = 8'b11101111; // 1110 * 1111
                mul = 0;

                #50

                mul = 1;

                #50 in = 8'b01101001; // 0110 * 1001
                mul = 0;

                #50

                mul = 1;

                #50

                #10 $stop;
        end
        initial $monitor($time, "in=%b␣mul=%b␣out=%b", in, mul, out);
endmodule
```

2.1.5 Xcelium Simulation Results:

Test Case 1 & 2:

Test Case 3 & 4:

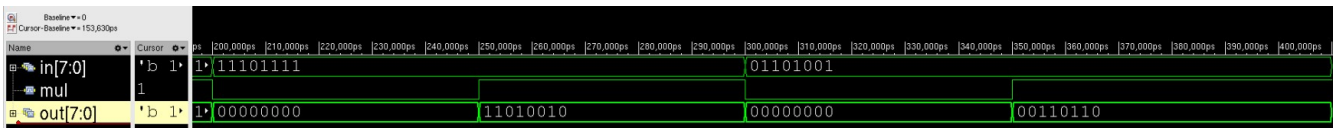


2.1.6 Genus Synthesis:

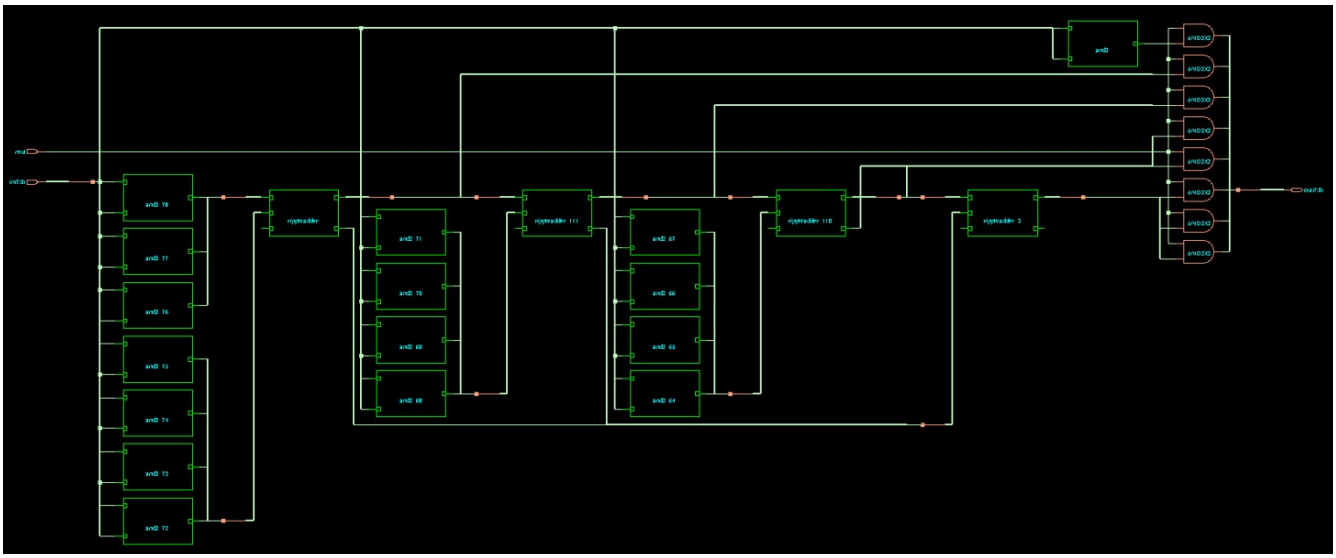
Simulations:

Test Case 1 & 2:

Test Case 3 & 4:



Standard Cell Schematic:



2.1.7 Timing and Power Summary:

Delay:

Maximum Delay: 6.13 ns

Power:

Component	Power (W)	Percentage
Internal Power	2.14×10^{-5}	34.71%
Switching Power	4.02×10^{-5}	65.11%
Leakage Power	$\mathbf{1.07 \times 10^{-7}}$	**0.17%**
Dynamic Power	$\mathbf{6.16 \times 10^{-5}}$	**99.82%**
Total Power	$\mathbf{6.17 \times 10^{-5}}$	**100%**

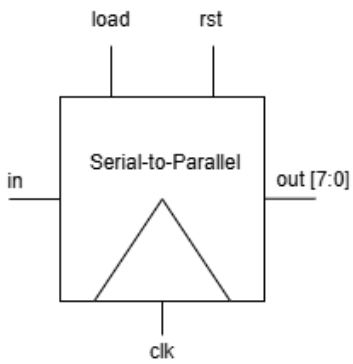
Table 1: 4x4 Multiplier Power Specifications

2.2 Serial-to-Parallel

2.2.1 Module Description:

A module that converts a stream of 8 serial input bits into a parallel 8-bit output. This module receives one bit at a time through the input **in**, with the first received bit corresponding with the most significant bit (MSB) of the final output. Each clock cycle shifts the bits left until the **load** input signal is high which produces an 8-bit output of the shifted bits.

2.2.2 Symbol:



2.2.3 Verilog Module:

```
module serialparallel ( input in, load, rst, clk,
                        output reg [7:0] out);

        reg [7:0] shift;

        always @(posedge clk or negedge rst) begin

                if (rst == 0) begin
                        shift <= 8'b00000000;
                        out <= 8'b00000000;
                end

                else begin
                        shift <= shift << 1;
                        shift[0] <= in;

                        if (load == 1) begin
                        out <= shift;
                        end

                end
        end
endmodule
```

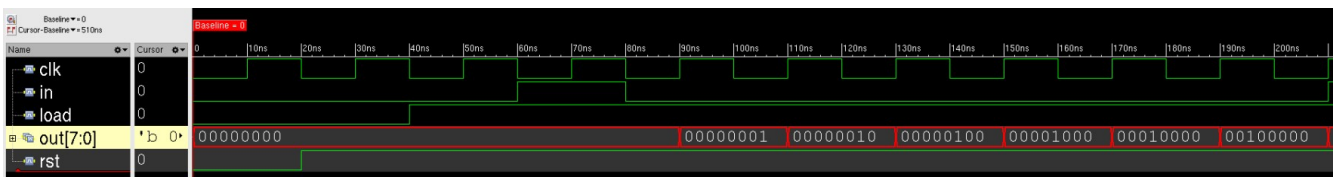
2.2.4 Verilog Testbench:

```
module SIPO_tb;
        reg in, rst, load, clk;
        wire [7:0] out;
        serialparallel DUT (.in(in), .load(load), .rst(rst), .clk(clk), .out(out));
        // 10 time - unit period clock
        always #10 clk = ~ clk;
        initial begin
                clk = 0;
                in = 0;
                load = 0;
                rst = 0; // active - low reset asserted
                // release reset
                #20 rst = 1;
                //load = 1
                #20 load = 1;

                // TEST 1: shifting of bits ( LSB -> MSB )
                #20 in = 1; // will be captured at next posedge
```

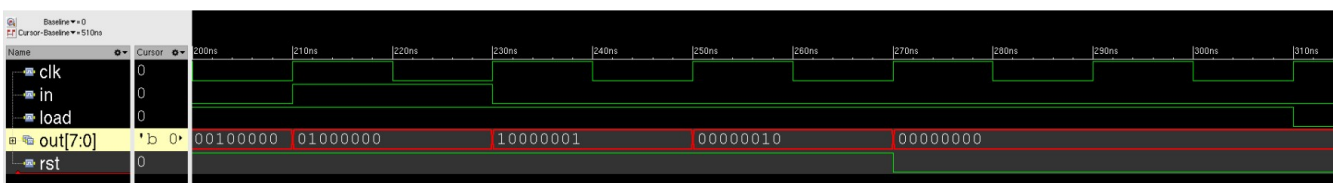
```
                #20 in = 0;
                #80;
                // TEST 2: reset back to 8     h00
                #30;
                #20 in = 1; // will be captured at next posedge
                #20 in = 0;
                #40 rst = 0;
                // TEST 3: load = 0
                #40 load = 0;
                #20 in = 1;
                #20 in = 0;
                #80
                #80 $finish;
        end
        initial begin
                $monitor ($time , "in␣=%b␣clk␣=%b␣rst␣=%b␣load␣=%b␣out␣=%b␣", in, rst, load, clk, out);
        end
endmodule
```

2.2.5 Xcelium Simulation Results:

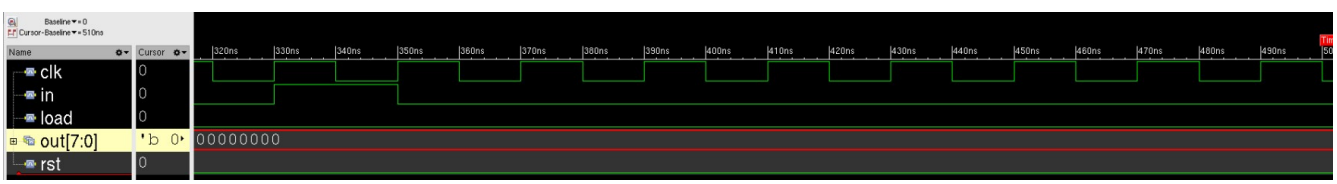
Test Case 1:



Test Case 2:



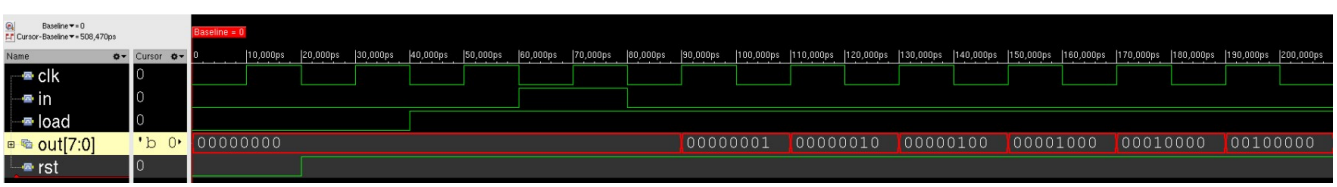
Test Case 3:



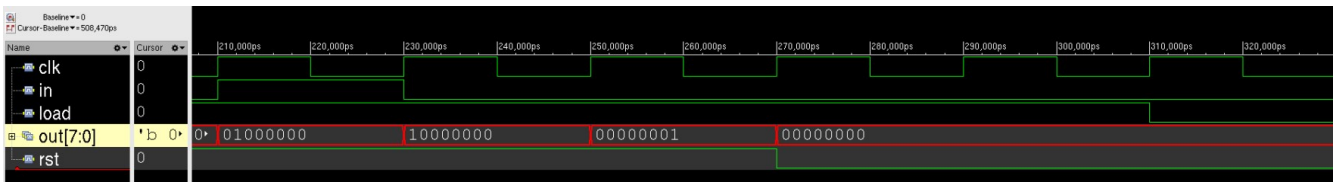
2.2.6 Genus Synthesis:

Simulations:

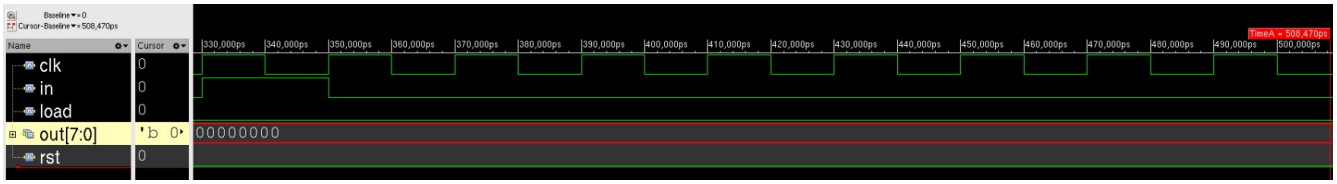
Test Case 1:



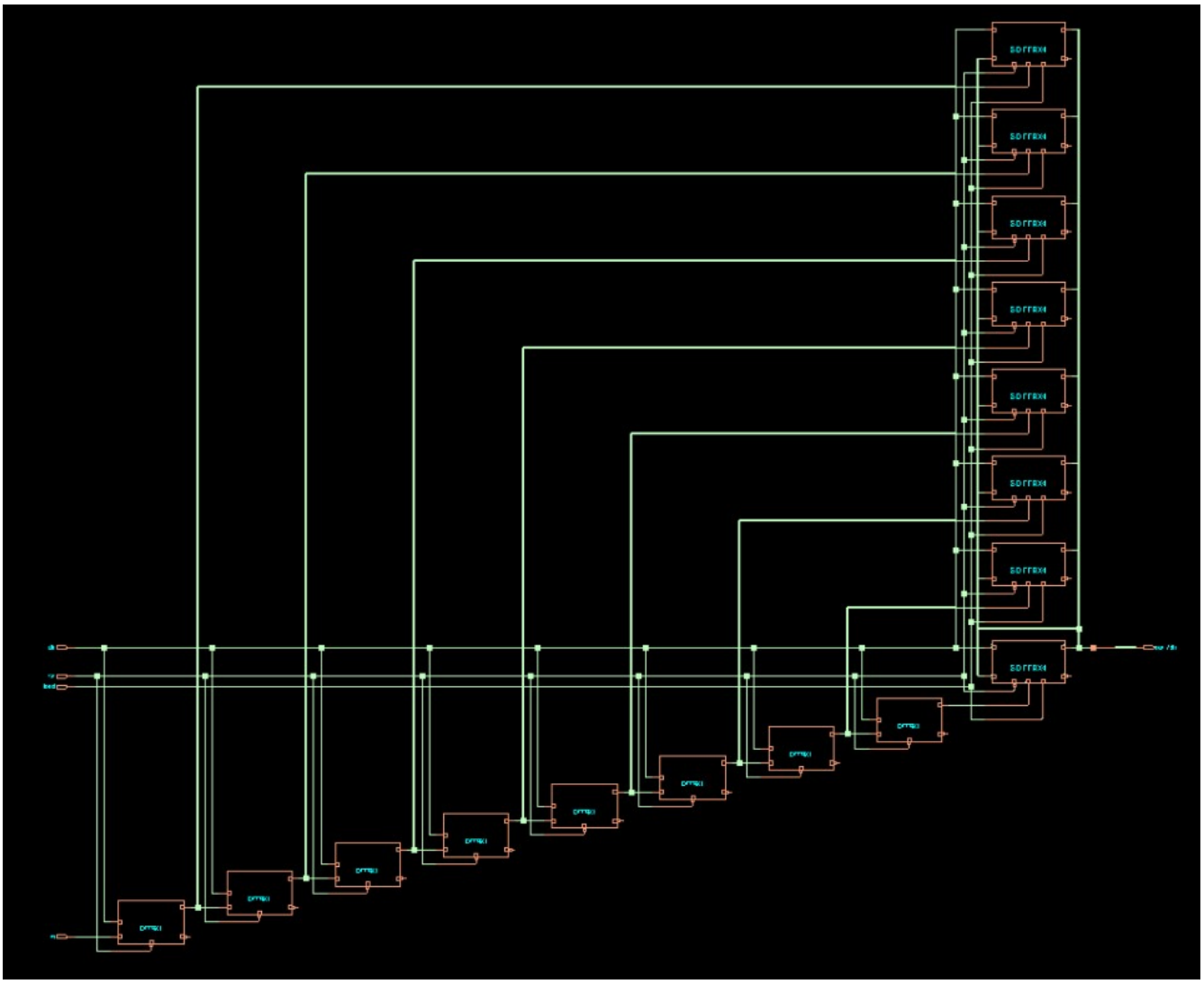
Test Case 2:



Test Case 3:



Standard Cell Schematic:



2.2.7 Timing and Power Summary:

Delay:

Maximum Delay: 1.041 ns
Slack: 4.624 ns

Power:

Component	**Power (W)**	**Percentage**
Internal Power	2.38×10^{-5}	39.57%
Switching Power	3.62×10^{-5}	60.18%
Leakage Power	$\mathbf{1.5 \times 10^{-7}}$	**0.25%**
Dynamic Power	$\mathbf{6 \times 10^{-5}}$	**99.75%**
Total Power	$\mathbf{6.01 \times 10^{-5}}$	**100%**

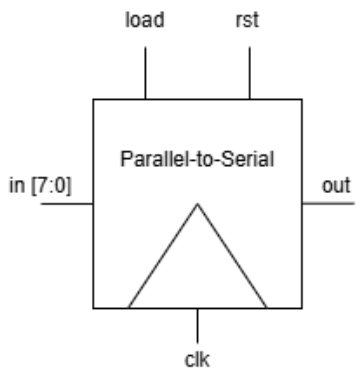
Table 2: Serial-to-Parallel Power Specifications

2.3 Parallel-to-Serial

2.3.1 Module Description:

A module that converts an 8-bit parallel input number into a stream of 8 serial output bits. This module receives one 8-bit number through the input **in**. Once the **load** input signal is high, each clock cycle displays the 8-bit input number one bit at a time.

2.3.2 Symbol:



2.3.3 Verilog Module:

```
module parallelserial ( input [7:0] in,
                        input load, rst, clk,
                        output reg out);

        reg [7:0] shift;

        always @(posedge clk or negedge rst) begin

                if (rst == 0) begin
                        shift <= 8'b00000000;
                        out <= 0;
                end

                else if (load == 1) begin
                        shift <= in;
                        out <= in[7];
                        end

                else begin
                        out <= shift[6];
                        shift <= shift << 1;
                        end

                end
endmodule
```

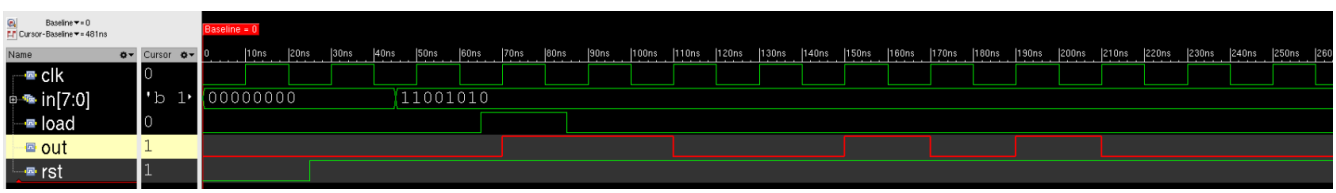
2.3.4 Verilog Testbench:

```
module parallelserial_tb;
        reg [7:0] in;
        reg clk, rst, load;
        wire out;

        initial clk = 0;
        always #10 clk = ~clk;

        parallelserial sp_00 (.in(in), .load(load), .rst(rst), .clk(clk), .out(out));

        initial begin
                rst = 0;
                in = 8'b00000000;
                load = 0;
```

```
            #25 rst = 1;

            #20 in = 8'b11001010; // Should display 11001010 then reset to 0

            #20 load = 1;
            #20 load = 0;

            #160

            #20 rst = 0;
            #20 rst = 1;

            #20

            #20 in = 8'b10101010; // Should display 10101010

            #20 load = 1;
            #20 load = 0;

            #160

            #20 rst = 0;
            #20 rst = 1;

            #20

            #20 in = 8'b11001011; // Should display 11001011

            #20 load = 1;
            #20 load = 0;

            #160

            #10 $stop;
        end
        initial $monitor($time, "␣in=%b␣clk=%b␣rst=%b␣load=%b␣out=%b", in, clk, rst, load, out);
endmodule
```

2.3.5 Xcelium Simulation Results:

Test Case 1:



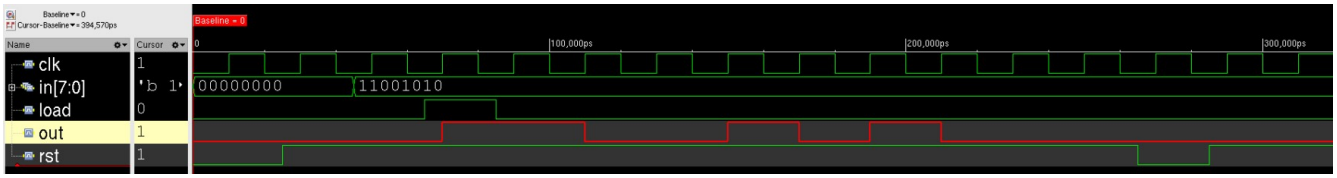
Test Case 2:

Test Case 3:

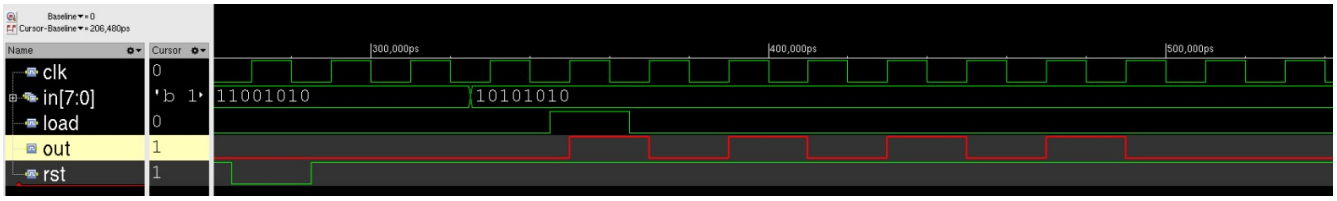
2.3.6 Genus Synthesis:

Simulations:

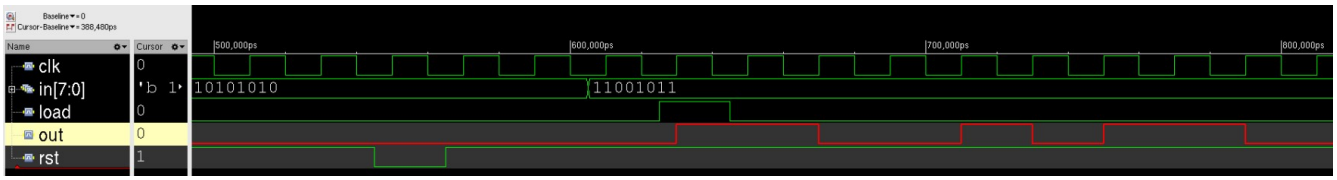
Test Case 1:



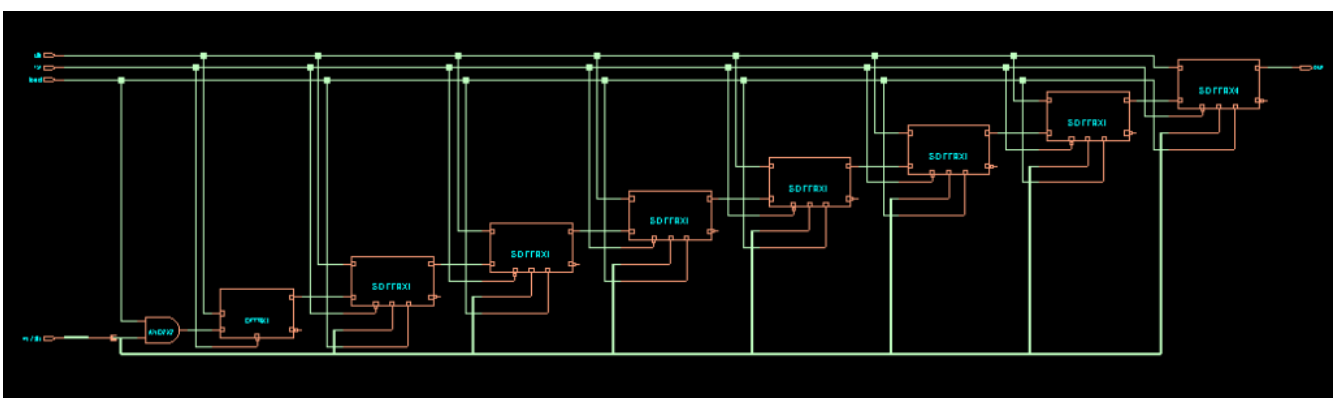
Test Case 2:



Test Case 3:



Standard Cell Schematic:



2.3.7 Timing and Power Summary:

Delay:

Maximum Delay: 748 ps
Slack: 9 ns

Power:

Component	**Power (W)**	**Percentage**
Internal Power	4.52×10^{-6}	43.67%
Switching Power	5.76×10^{-6}	55.64%
Leakage Power	$\mathbf{7.16 \times 10^{-8}}$	**0.69%**
Dynamic Power	$\mathbf{1.03 \times 10^{-5}}$	**99.31%**
Total Power	$\mathbf{1.04 \times 10^{-5}}$	**100%**

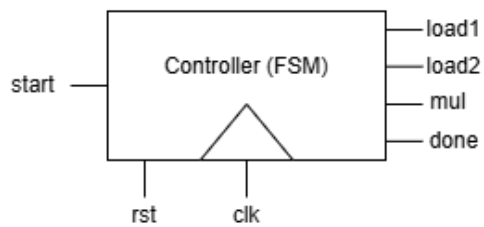
Table 3: Parallel-to-Serial Power Specifications

2.4 Controller (FSM)

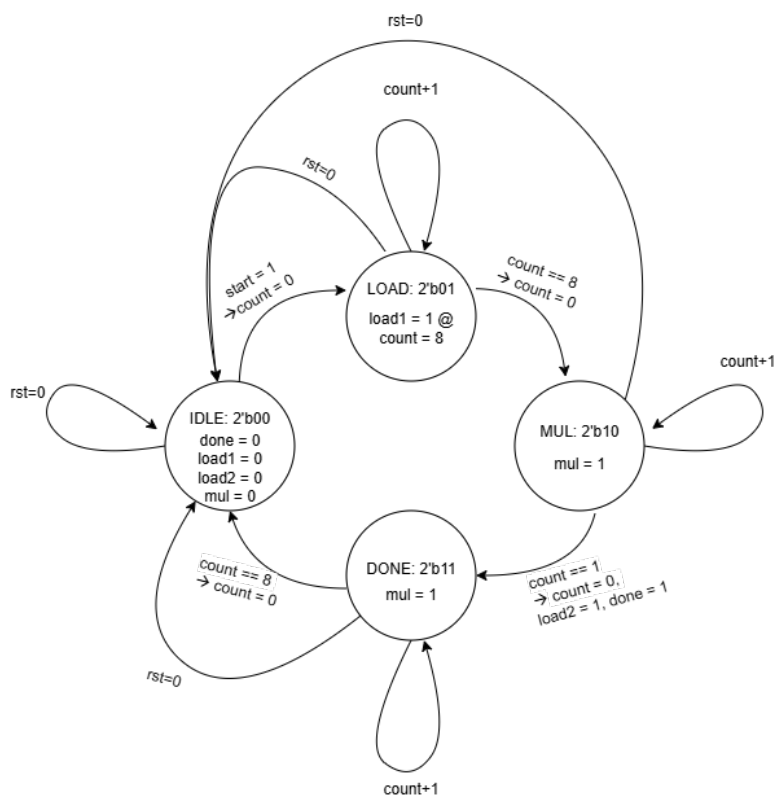
2.4.1 Module Description:

A Moore finite state machine (FSM) module responsible for coordinating the entire system. The design has 4 states with a 4-bit counter integrated into it. The only user input is **start** which moves the FSM out of the idle state. State (01) waits 8 clock cycles then the **load1** signal becomes high for the serial-to-parallel output and goes onto the next state. State (10) sends the **mul**, **done** and **load2** signal high for the multiplier to produce it's output, for the done flag to raise, and the parallel-to-serial to receive it's input, then goes onto the next state. The last state (11) waits for 8 seconds to make sure all 8 streamed bits are all produced in the output and then the FSM goes back into idle (00).

2.4.2 Symbol:



2.4.3 State Diagram:



2.4.4 Verilog Module:

```
module controllerfsm (  input clk, rst, start,
                        output reg done, mul, load1, load2);

        reg [1:0] state, next_state;
        reg [3:0] count;


        always @(posedge clk or negedge rst) begin

                if (rst == 0) begin
                state <= 2'b00;
                count <= 4'b0000;
                end

                else begin
                        if (state != next_state)
                        count <= 4'b0000;

                        else
                        count <= count + 1;

                state <= next_state;

                end

        end

        always @(*) begin

                next_state = state;
                load1 = 0;
                load2 = 0;
                done = 0;
                mul = 0;

                case (state)

                        2'b00: begin
                                if (start == 1)
                                next_state = 2'b01;
                                end

                        2'b01: begin
                                if (count == 4'b1000) begin
                                load1 = 1;
                                next_state = 2'b10;
                                end
                                end

                        2'b10:  begin
                                mul = 1;
                                if (count == 4'b0001) begin
                                load2 = 1;
                                done =1;
                                next_state = 2'b11;
                                end
                                end

                        2'b11:  begin
                                mul = 1;
                                if (count == 4'b1000) begin
                                next_state = 2'b00;
                                end
                                end

                endcase
        end
endmodule
```

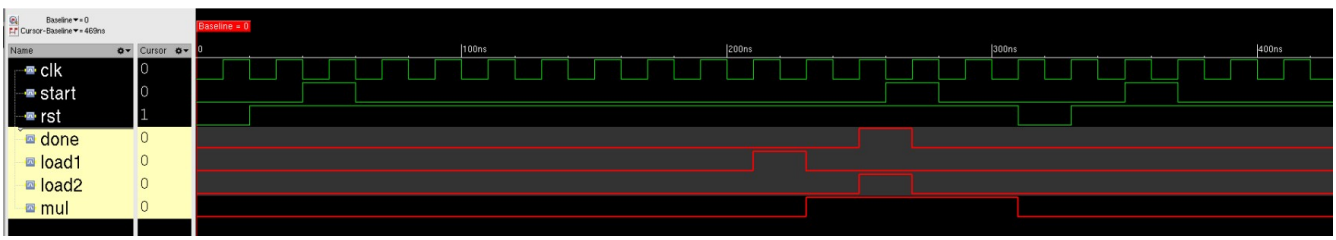
2.4.5 Verilog Testbench:

```
module controllerfsm_tb;
        reg clk, rst, start;
        wire done, mul, load1, load2;
        initial clk = 0;
        always #10 clk = ~clk;
        controllerfsm fsm_00 (.clk(clk), .rst(rst), .start(start), .done(done), .mul(mul), .load1(load1), .load2
             (load2));
        initial begin
            // reset behavior
```

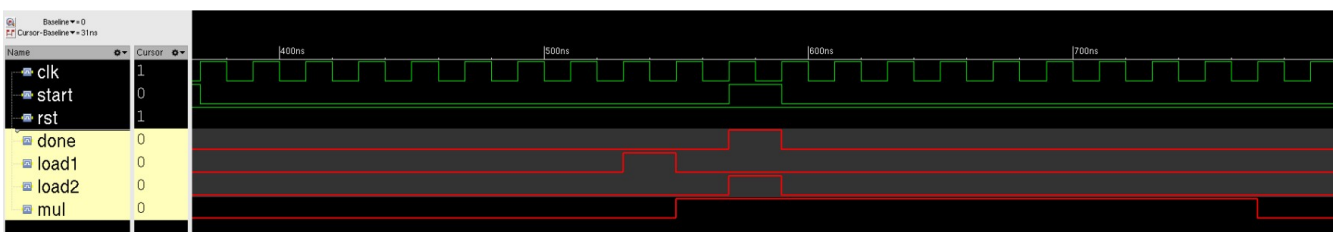
```
            rst = 0; start = 0;
            #20 rst = 1;

            // TEST 1: normal operation
            #20 start = 1;
            #20 start = 0;
            #200;

            // TEST 2: reset mid-operation
            start = 1;
            #20 start = 0;
            #30 rst = 0;
            #20 rst = 1;
            #20;

            // TEST 3: back to back operations
            start = 1;
            #20 start = 0;
            #200;
            start = 1;
            #20 start = 0;
            #200;

            #10 $stop;
        end
        initial $monitor($time, "␣clk=%b␣rst=%b␣start=%b␣done=%b␣mul=%b␣load1=%b␣load2=%b", clk, rst, start,
            done, mul, load1, load2);
endmodule
```

2.4.6 Xcelium Simulation Results:

Test Cases 1 & 2:



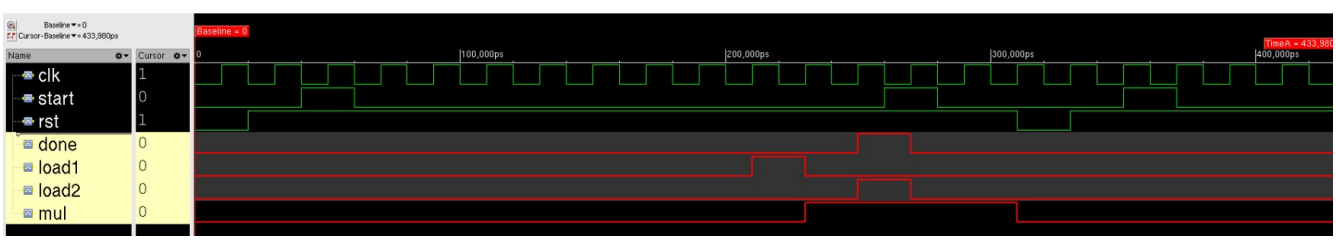
Test Case 3:



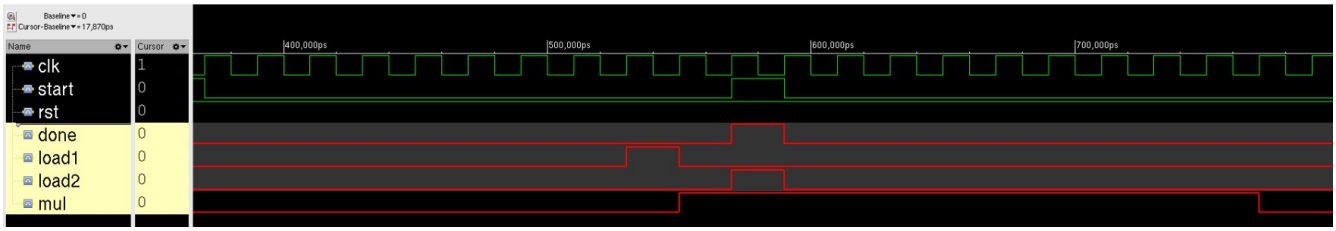
2.4.7 Genus Synthesis:

Simulations:

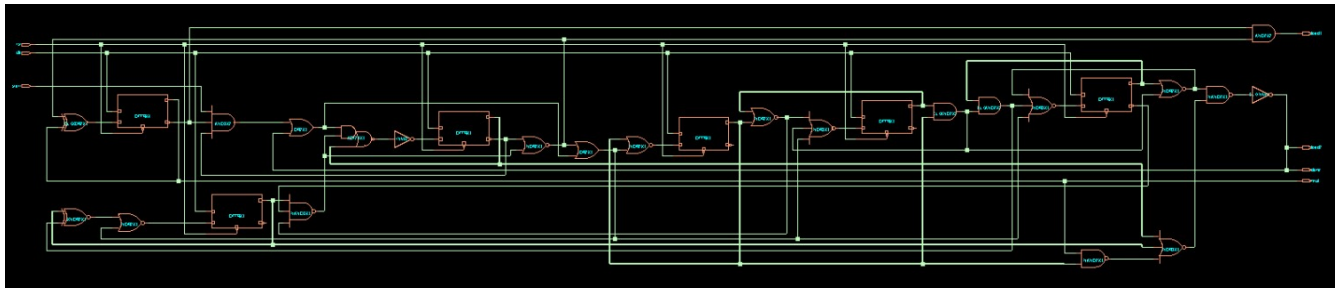
Test Cases 1 & 2:



Test Case 3:



Standard Cell Schematic:



2.4.8 Timing and Power Summary:

Delay:

Maximum Delay: 2.89 ns
Slack: 3.08 ns

Power:

Component	Power (W)	Percentage
Internal Power	1.99×10^{-5}	44.19%
Switching Power	2.50×10^{-5}	55.66%
Leakage Power	$\mathbf{6.45 \times 10^{-8}}$	**0.14%**
Dynamic Power	$\mathbf{4.49 \times 10^{-5}}$	**99.85%**
Total Power	$\mathbf{4.49 \times 10^{-5}}$	**100%**

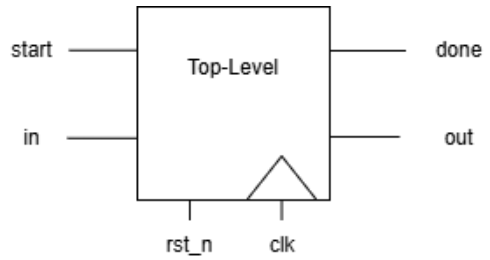
Table 4: FSM Power Specifications

2.5 Top-Level Implementation

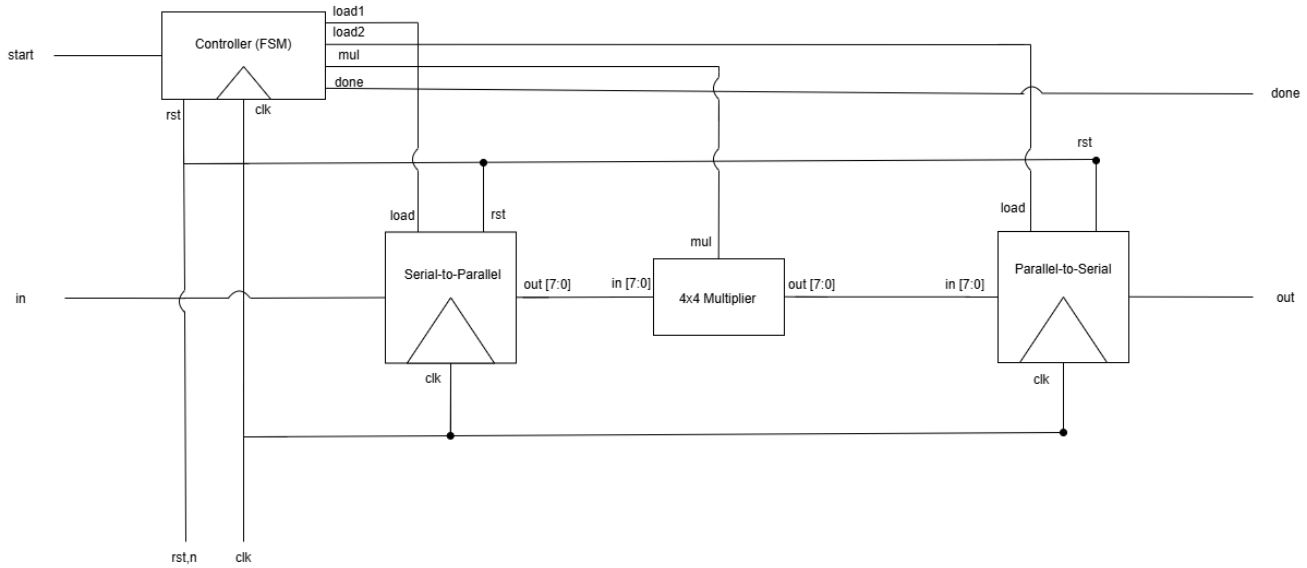
2.5.1 Module Description:

The top-level module integrates the controller FSM, serial-to-parallel converter, 4-bit multiplier, and parallel-to-serial converter to create a processing system. Firstly, the **start** input is raised by the user for the FSM to start controlling the system. The input **in** then takes 8 clocked 1-bit inputs which goes to the serial-to-parallel module for the multiplier to receive it's 8-bit input. Once the FSM sends the parallel-to-serial it's load signal in order for it to take in the multiplier output, the **done** output flag is raised by the FSM while producing it's serial 8-bits for 8 cycles after that.

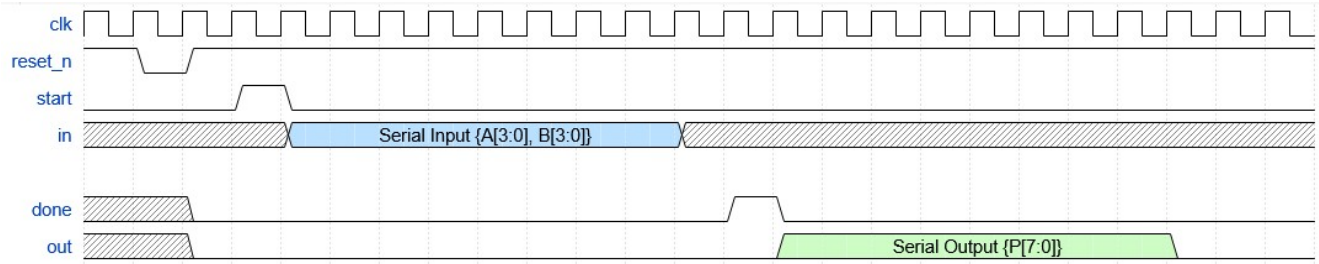
2.5.2 Symbol:



2.5.3 Block-Level Diagram:



2.5.4 Timing Diagram:



2.5.5 Verilog Module:

```
module project (input in, start, rst, clk,
                output out, done);

        wire [7:0] o_sp, o_mp4;
        wire i_mul, i_load1, i_load2;

        controllerfsm cfsm_00 (.clk(clk), .rst(rst), .start(start), .done(done), .mul(i_mul), .load1(i_load1), .
            load2(i_load2));
        serialparallel sp_00 (.in(in), .load(i_load1), .rst(rst), .clk(clk), .out(o_sp));
        multiplier4 mp4_00 (.in(o_sp), .mul(i_mul), .out(o_mp4));
        parallelserial ps_00 (.in(o_mp4), .load(i_load2), .rst(rst), .clk(clk), .out(out));

endmodule
```

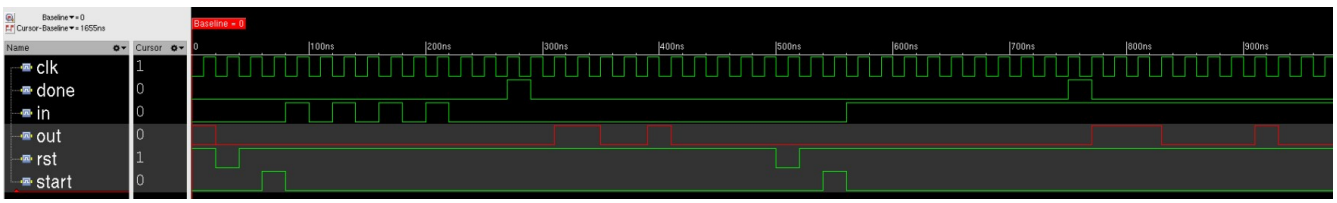
2.5.6 Verilog Testbench:

```
module project_tb;

        reg in, start, rst, clk;
        wire out, done;

        initial clk = 0;
        always #10 clk = ~clk;

        project proj_00 (.in(in), .start(start), .rst(rst), .clk(clk), .out(out), .done(done));
        initial begin
                rst = 1;
                start = 0;
                in = 0;


                #20 rst = 0;
                #20 rst = 1;

                #20 start = 1;
                #20 start = 0;
                in = 1;
                #20 in = 0;
                #20 in = 1;
                #20 in = 0;
                #20 in = 1;
                #20 in = 0;
                #20 in = 1;
                #20 in = 0;

                #260

                #20 rst = 0;
                #20 rst = 1;

                #20 start = 1;
                #20 start = 0;
                in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;

                #260

                #20 rst = 0;
                #20 rst = 1;

                #20 start = 1;
                #20 start = 0;
                in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 0;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;
                #20 in = 1;

                #260

                #20 rst = 0;
```

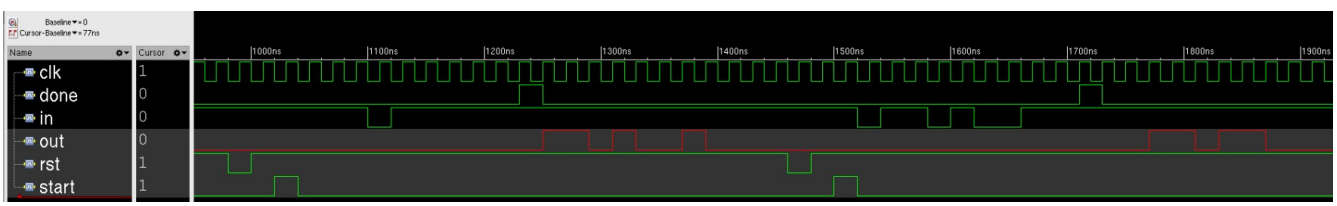
```
            #20 rst = 1;

            #20 start = 1;
            #20 start = 0;
            in = 0;
            #20 in = 1;
            #20 in = 1;
            #20 in = 0;
            #20 in = 1;
            #20 in = 0;
            #20 in = 0;
            #20 in = 1;

            #260

            #10 $stop;
        end
        initial $monitor($time, " in=%b clk=%b rst=%b start=%b out=%b done=%b, sp_00_out=%b", in, clk, rst,
            start, out, done, proj_00.sp_00.out);
endmodule
```

2.5.7 Xcelium Simulation Results:

Test Case 1 & 2:



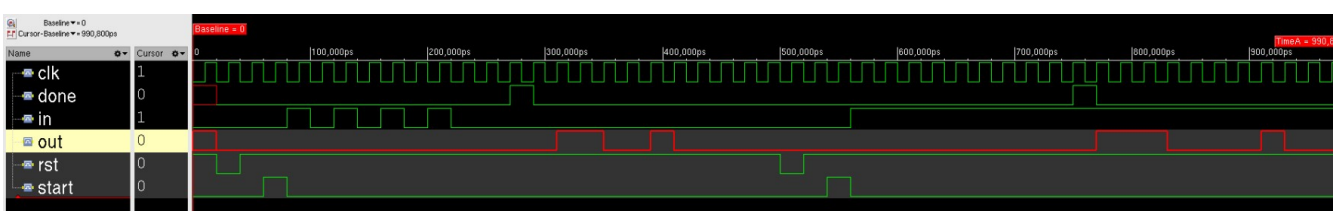
Test Case 3 & 4:



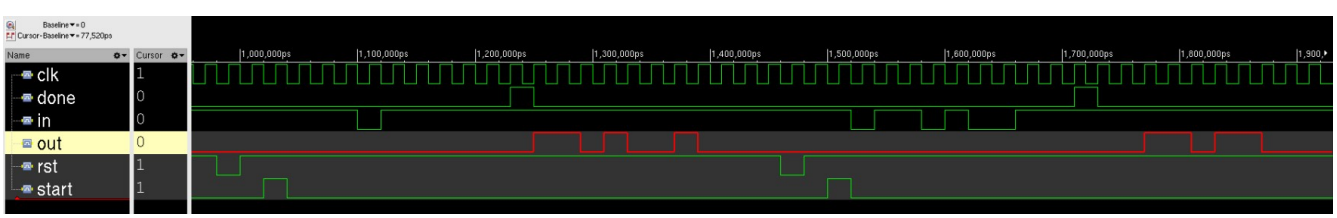
2.5.8 Genus Synthesis Results:

Simulations:

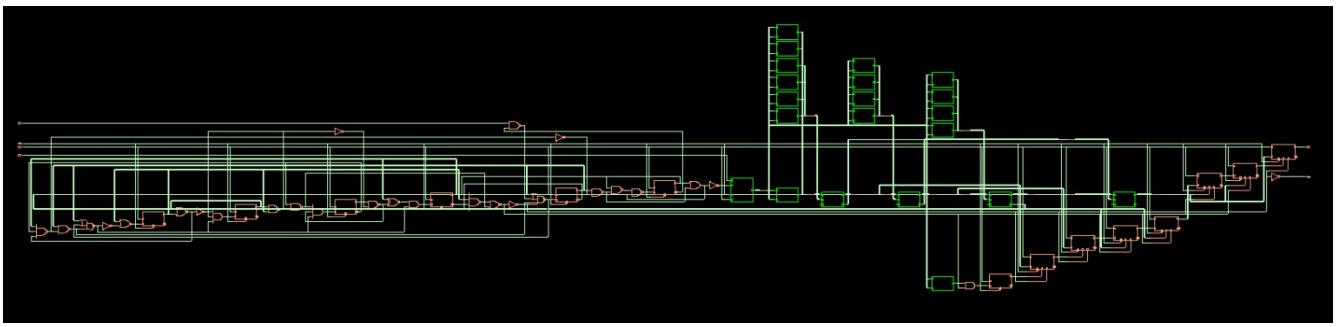
Test Case 1 & 2:



Test Case 3 & 4:



Standard Cell Schematic:



2.5.9 Timing and Power Summary:

Delay:

Maximum Delay: 5.73 ns
Slack: 12 ps

Max Frequency:
The max frequency of the whole system is set by the single worst top-level critical path, not the sum of all block critical paths. The estimated max frequency is based off of critical path of the multiplier since its path has the most delay.

$$f_{max,estimate} = \frac{1}{T_{CriticalPath,mul}} = \frac{1}{6.13ns} = 163.1MHz$$

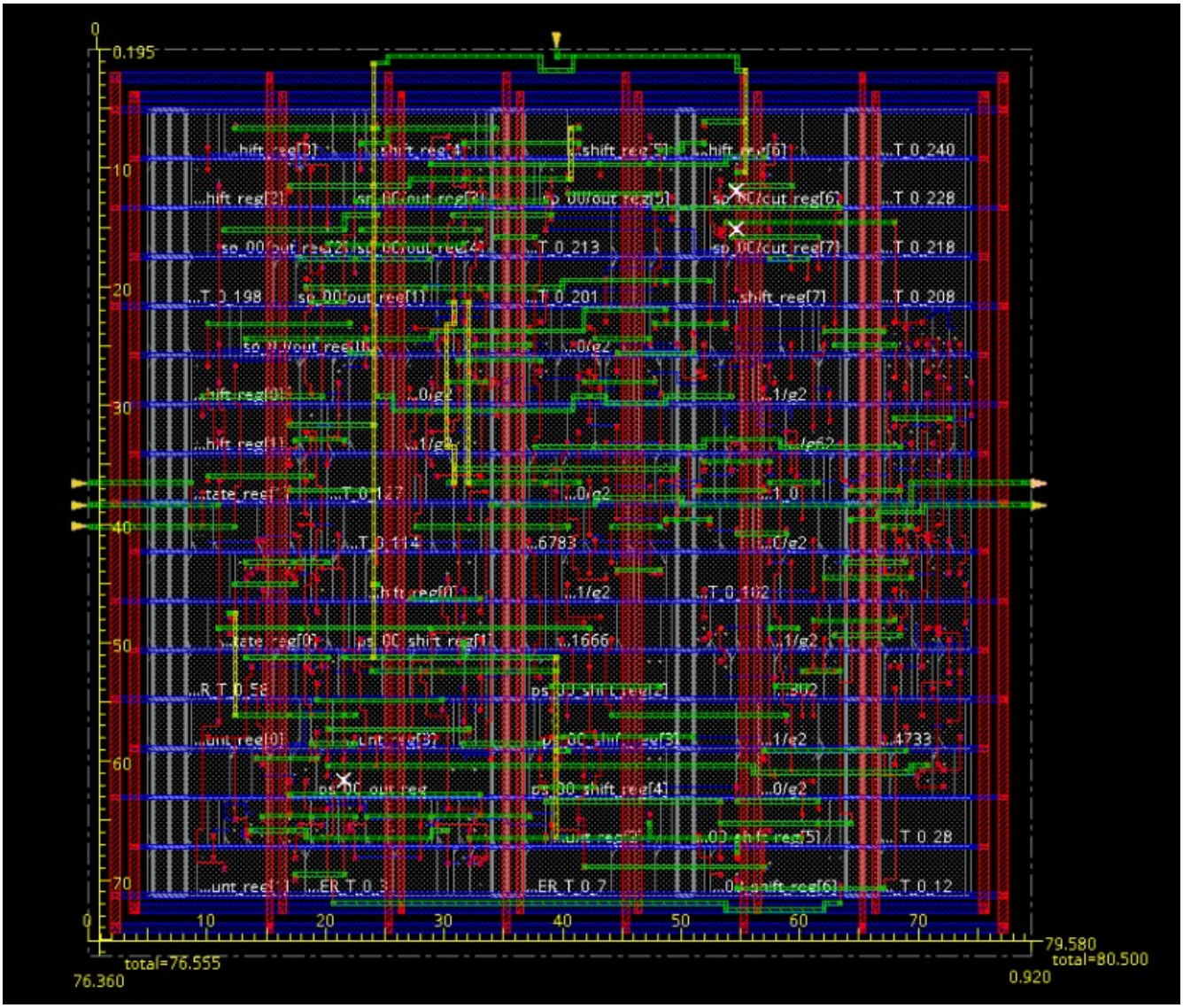
$$f_{max,actual} = \frac{1}{T_{CriticalPath}} = \frac{1}{5.73ns} = 174.5MHz$$

Power:

Component	**Power (W)**	**Percentage**
Internal Power	7.87×10^{-5}	51.52%
Switching Power	7.37×10^{-5}	48.24%
Leakage Power	$\mathbf{3.65 \times 10^{-7}}$	**0.24%**
Dynamic Power	$\mathbf{1.52 \times 10^{-4}}$	**99.77%**
Total Power	$\mathbf{1.53 \times 10^{-4}}$	**100%**

Table 5: Top-Level Multiplier Power Specifications

2.5.10 Place and Route:



The total final required design area is calculated as:

$$A = 76.555\ \mu m \times 80.5\ \mu m = 6162.68\ \mu m^2$$

2.6 Design Decisions & Functionality

The overall system was designed as a multi-cycle serial multiplier to reduce hardware complexity while maintaining correct functionality. A serial approach was selected where input data is received one bit at a time, computed in parallel, the result is then transmitted serially once again. This design choice allows for simplicity and re-usability. The design utilizes a finite state machine controller to coordinate the overall operation, controlling when data is loaded, when to multiply, and when results are done computing. A counter is built in to enforce a fixed cycle operation during input and output phases, ensuring correct timing and module synchronization.

Overall, the design emphasizes a modular and structured approach. Separation of control and data-path to simplify debugging and verification. Design choices were made to prioritize a balance of performance and correctness. This balance allows for reduced hardware complexity and showcases core digital design principles.

2.7 Testing & Ensuring Functionality

4-bit Multiplier:
To ensure the 4-bit multiplier worked as intended, multiple 8-bit inputs with the first 4-bits being one operand and the last 4-bits being the second operand were put into the system. These inputs were chosen to test if all the carries and all the output bits functionality worked correctly. Then we verified the correct product would output only when the **mul** input signal went high for each case. This was to ensure that the FSM had full control over when to send the input and outputs of the multiplier.

Serial-to-Parallel:
To ensure the serial-to-parallel module worked as intended, multiple 1-bit inputs were clocked into the input with the **load** signal high. This was to verify that the bits were shifting correctly into the 8-bit output. Next, the **rst** input signal was checked to be active-low by activating it in the middle of shifting bits. Lastly, we made sure there would be no output when **load** stayed low, even with clocked 1-bit inputs.

Parallel-to-Serial:
To ensure the parallel-to-serial module worked as intended, multiple 8-bit inputs were sent into the input. Once an 8-bit input was put into the module, the **load** signal goes high so that we can verify that the same 8-bit input becomes individually clocked at the output with the MSB being the first bit.

Finite State Machine:
To ensure the FSM worked as intended, the **start** input would be activated and we verified it would wait 8 clock cycles before the **load1** signal activated in order to take in 8-bits at the input of the serial-to-parallel module. Once the **load1** signal activated, the **mul** signal should be activated directly after so that the multiplier receives the serial-to-parallel output and provides the calculation to the parallel-to-serial module. After the **mul** signal is activated, the **load2** and **done** signal should activate in order to let to the user know that the product is ready to start outputting. Next, there's a test case that activates the **rst** signal to ensure the whole fsm resets when activated. Lastly, the last test case activates the start signal again to make sure that the FSM can work in back-to-back inputs and resets.

Top-Level:
To ensure the top-level module worked as intended, all the sub-modules (4-bit multiplier, serial-to-parallel, parallel-to-serial, and FSM) were combined and tested together utilizing multiple 1-bit clocked inputs that created an 8-bit number which was split and used for the multiplier. The same operands used to test the multiplier module were used to ensure the output could handle carries and for all ouput bits to be tested. The **rst** and **clk** signals were applied to all the modules except the multiplier because it does not need them. After the **start** signal was activated, the serial 8-bit numbers were applied to the **in** signal. The **done** flag would raise with the output of the product being serially outputted with the MSB first. The **rst** signal was activated in between test cases to ensure that the it worked as intended and for the system to return to a known initial state before the next test case. Overall, each test cased produced an 8-bit clocked output that was the correct product of the two 4-bit numbers being multiplied.

2.8 Challenges

The primary challenge encountered during this project has been the design of the Finite state machine controller. the primary issue faced with controller has been ensuring correct sequencing of control signals like **load1**, **load2**, **mul**, and **done**. These control signals required careful coordination with core blocks.

Another challenge faced was the implementation of ripple-carry adders to perform multiplication operations. Ensuring carry propagation across multiple stages was particularly challenging in achieving the desired functionality.

The final major issue encountered was post-synthesis timing behavior introduced. Post-synthesis simulation showed flaws in static timing analysis particularly in setup and hold violations, which we overcame by tweaking the FSM controller to sequence control signals more efficiently.

2.9 Conclusion Statements

If given the chance to design this serial-to-serial multiplier from scratch again. We would design modules with performance in mind, minimizing redundancy to further eliminate critical path delay.

Additionally, having a clear and structured approach when figuring out the design of blocks to improve readability and simplify the debugging process.

Finally, getting Modules synthesized earlier to better understand timing constraint. Doing this will give us more time figuring out which blocks need to be optimized leading to overall better designs.